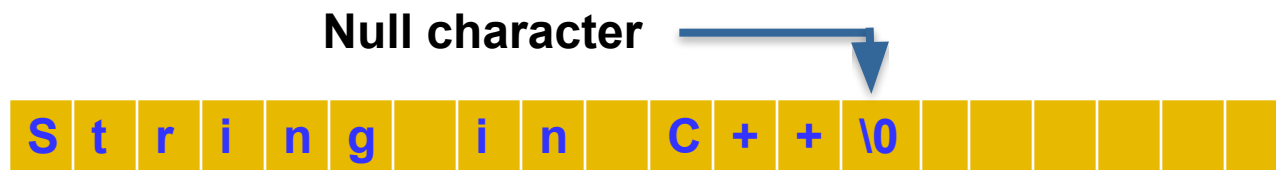


String Handling in C++

1. String

- *A collection of characters written in double quotations is called **string** or **string constant**.* It may consist of any alphabetic characters, digits, and special symbols.
- *A string is stored as an array of characters.* The array is terminated by a special symbol known as **null character**. It is denoted by **\0** and is used to indicate the end of string. It consists of backslash and zero.



- The values stored in an array of characters can be manipulated individually using the index of the array. The user can *input, process* and *displays* the individual characters of the string in the same way as an array. **Examples:** some examples of strings are as follows:

“Pakistan”

“123”

“99-Mall Road, Lahore”

1.1 String Declaration

C++ stores a string as an array of characters. An array is a group of consecutive memory locations that can store same type of data.

Syntax

The syntax of declaring a string in C++ is as follows:

char array_name [length];

char: It indicates the type of array used as string.

Array_name : It indicates the name of array.

length: It indicates the number of memory locations in the array.

1.1 String Declaration (continued...)

Example: An example of string declaration is as follows:

```
char book [20];
```

The above statement declares a variable **book**. The value 20 in brackets indicates the length of string that can be stored in the variable. The above declaration allocates twenty memory locations and each memory location can store one character as follows:



1.2 String Initialization

Syntax

The syntax of initializing a string in C++ is as follows:

char array_name [length] = value;

char: It indicates the type of array used as string.

Array_name : It indicates the name of array.

length: It indicates the number of memory locations in the array.

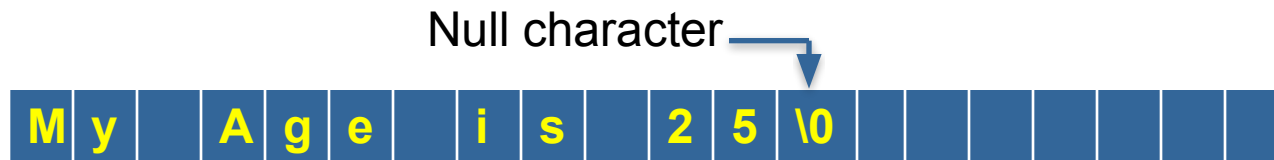
value: It indicates the value initialized in string. It is enclosed in double quotes.

1.2 String Initialization (continued...)

Example: An example of string initialization is as follows:

```
char age [20] = " My Age is 25";
```

The above statement will store the string as follows:



The last character in the memory is known as **null character**. It consists of back slash and zero. It indicates the end of string. It is added at the end of string **automatically**. *A string variable can also be declared and initialized without indicating the length as follows:*

```
char name [ ] = " Pakistan";
```



The above statement automatically declares a string according to the length of value.

1.2 String Initialization (continued...)

Program 1: Write a program that displays the value stored in a string variable *name* .

```
#include<iostream>
using namespace std;
int main()
{
    char age[ ] = "My Age is 25";
    cout<<age;
    return 0;
}
```

Output

A screenshot of a terminal window with a black background. The text "My Age is 25" is displayed in a light gray monospace font. Below the text is a horizontal dashed line of the same color.

2. String Input

- *A string can be input from the user using different functions.* C++ provides the following functions for string input:

C++ Routine	Description
cin	General purpose keyboard input
cin.getline()	Input a string of characters from keyboard
cin.get()	Input a single character from keyboard

2.1 The 'cin' Object

*The **cin** object is used to input a string value without any blank space in it. It does not support a string with spaces. If the user inputs a string value that contains any space, the value before the space will be used. The remaining string value will be ignored.*

Syntax

The syntax of getting input using **cin** object is as follows:

cin>>str;

str: It indicates the name of string variable in which the value is to be stored.

2.1 The 'cin' Object (continued...)

Program 2: Write a program that inputs the name of the user and displays it using **cin** operator.

```
#include<iostream>
using namespace std;
int main()
{
    char name[50];
    cout<<" Enter your name: ";
    cin>>name;
    cout<<" Your name: "<<name<<endl;
    return 0;
}
```

Output

```
Enter your name: Tehseen Ahsan
Your name: Tehseen
-----
```

2.1 The 'cin' Object (continued...)

How Program 2 works?

The program uses *cin* object to input the name of the user. The user inputs “ Tehseen Ahsan” . The *cin* object cannot input a string with blank space. It only stores the value before space. The output displays only “Tehseen”.

2.2 cin.getline()

*The **getline** function of **cin** object is used to input any string value including blank spaces. The user can input any type of string value.*

Syntax

The syntax of using this function is as follows:

cin.getline(str , len);

str: It indicates the name of string variable in which the value is to be stored.

len: It indicates the length of string variable.

2.2 cin.getline() (continued...)

Program 3: Write a program that inputs the name of the user and displays it using **cin.getline()** function.

```
#include<iostream>
using namespace std;
int main()
{
    char name[50];
    cout<<" Enter your name: ";
    cin.getline(name,50);
    cout<<" Your name: "<<name<<endl;
    return 0;
}
```

Output

```
Enter your name: Tehseen Ahsan
Your name: Tehseen Ahsan
-----
```

2.3 cin.get()

*The **get()** function of **cin** object is used to input a single character.*

Syntax

The syntax of using this function is as follows:

cin.get(str);

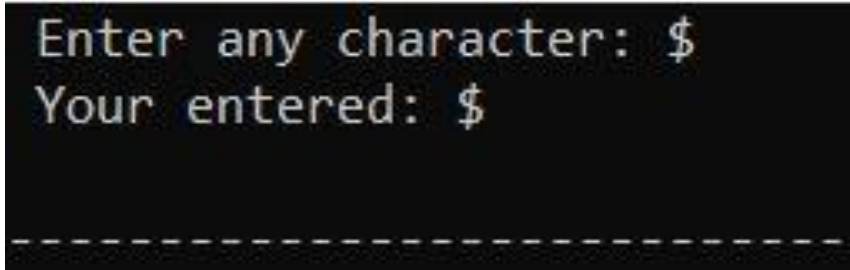
str: It indicates the name of character variable in which the value is to be stored.

2.3 cin.get() (continued...)

Program 4: Write a program that inputs a character from the user and displays it on screen using **cin.get()** function.

```
#include<iostream>
using namespace std;
int main()
{
    char str;
    cout<<" Enter any character: ";
    cin.get(str);
    cout<<" Your entered: "<<str<<endl;
    return 0;
}
```

Output

A screenshot of a terminal window with a black background and white text. It shows the program's output: "Enter any character: \$" followed by "Your entered: \$" on the next line. A dashed line is visible at the bottom of the terminal window.

```
Enter any character: $
Your entered: $
```

2.4 Miscellaneous Programs

Program 5: Write a program that inputs a string from the user and displays its length.

```
#include<iostream>
using namespace std;
int main()
{
    char str[50];
    int i = 0;
    cout<<" Enter a string: ";
    cin.getline(str, 50);
    while(str[i]!='\0')
        i++;
    cout<<" The length of string is " <<i<<endl;
    return 0;
}
```

Output

```
Enter a string: String Handling using C++
The length of string is 25
```

2.4 Miscellaneous Programs

Home Work Assignment:

Program 6: Write a program that inputs a string from the user and then copies it in another string.

Program 7: Write a program that inputs a string from the user and counts the number of vowels in the string.

3. Array of Strings

An array of strings is actually *a two dimensional array of characters*. *Each row of the array represents one string*. Each character in an array of strings is stored in a separate index of a two dimensional array.

Syntax

The syntax of declaring an array of strings is as follows:

char str [rows][cols];

str: It indicates the name of two dimensional array.

rows: It indicates the number of rows in an array.

cols: It indicates the numbers of columns in each row of an array.

Example: **char names[3][50];**

The above example declares a two dimensional array of characters with three rows and fifty columns in each row. *It can be considered as an array of three strings*.

** Number of characters in an array = no. of rows x no. of columns.*

3.1 Initializing Array of Strings

An array of strings can be initialized in different ways. It can be initialized *by assigning individual characters to each index in the array*. It can also be initialized *by assigning complete string to each row in the array*.

Example

The following example initializes an array of strings *by assigning individual characters*:

```
char str[3][5] = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'm', 'o'};
```

The above example declares a two-dimensional array of characters and initializes it with the given values as follows:

	0	1	2	3	4
0	a	b	c	d	e
1	f	g	h	i	j
2	k	l	m	n	o

3.1 Initializing Array of Strings (continued...)

Another method of initializing an array of strings is as follows:

```
char str[3][10] = {"Ali", "Abdullah", "Usman"};
```

The above example declares two dimensional array of characters with three rows and ten columns in each row. The string values are assigned to different rows (to 3 rows) in the array as follows:

	0	1	2	3	4	5	6	7	8	9
Str [0]	A	i	i	\0						
Str [1]	A	b	d	u	l	l	a	h	\0	
Str [2]	U	s	m	a	n	\0				

The above example shows that each string is automatically terminated with the null character. The array contains string values in three rows.

** Number of characters space in above example: $3 \times 10 = 30$.*

3.2 Input/Output with Array of Strings

Each row of a two-dimensional array of characters represent a complete string. The values can be input to an array of string by referencing the complete row of a two dimensional array of characters as follows:

```
char str[3][10];  
cin>>str[0];      // represents the first row in the array  
cin>>str[1];      // represents the second row in the array  
cin>>str[2];      // represents the third row in the array
```

Similarly, the values can be displayed by references the complete row of a two dimensional array of characters as follows:

```
char str[3][10];  
cout<<str[0];     // represents the first row in the array  
cout<<str[1];     // represents the second row in the array  
cout<<str[2];     // represents the third row in the array
```

3.2 Input/Output with Array of Strings (continued...)

Program 8: Write a program that inputs the names of five cities and then displays them.

```
#include<iostream>
using namespace std;
int main()
{
    char city[5][30];
    for(int i=0;i<5;i++)
    {
        cout<<" Enter city name: ";
        cin>> city[i];
    }
    for(int i=0;i<5;i++)
    {
        cout<< city[i]<<endl;
    }
    return 0;
}
```

Output

```
Enter city name: Islamabad
Enter city name: Lahore
Enter city name: Karachi
Enter city name: Peshawar
Enter city name: Quetta
Islamabad
Lahore
Karachi
Peshawar
Quetta
-----
```

3.2 Input/Output with Array of Strings (continued...)

Home Work Assignment:

Program 9: Write a program that inputs the names of five countries. It only displays the countries whose name starts with a vowel.

3.3 String Functions (string.h)

The header file '**string.h**' contains the built-in functions that are used to process string values. Some important functions defined in this header file are as follows:

3.3.1 strcat()

The **strcat()** function is used to append a copy of one string to the end of second string. It also terminates the resulting string with null character. The string in which the values copied should have enough space to hold the result.

Syntax

strcat(str1, str2);

strcat: It is the name of the function.

str1: It indicates the name of string in which the value is to be appended.

str2: It indicates the name of string whose value is to be appended to str1.

3.3 String Functions (string.h) continued....

Example:

```
#include<iostream>
#include<string.h>
using namespace std;
int main()
{
char str1[50] = "The more you work....";
char str2[50] = "The more you gain....";
cout<<"String 1: ";
cout<<str1<<endl;
cout<<"String 2: ";
cout<<str2<<endl;
cout<<"Concatenate the strings...."<<endl;
strcat(str1,str2);
cout<<"String 1 after concatenation: ";
cout<<str1;
return 0;
}
```

Output

```
String 1: The more you work....
String 2: The more you gain....
Concatenate the strings....
String 1 after concatenation: The more you work....The more you gain....

-----
Process exited after 0.03794 seconds with return value 0
Press any key to continue . . .
```

3.3 String Functions (string.h) continued....

3.3.2 strncat()

The **strncat()** function is used to append the specified number of characters of one string to the end of second string. It also terminates the resulting string with null character. The string in which the values copied should have enough space to hold the result.

Syntax

strncat(str1, str2, n);

strncat: It is the name of the function.

str1: It indicates the name of string in which the value is to be appended.

str2: It indicates the name of string whose value is to be appended to str1.

n: It indicates the number of characters to be appended.

3.3 String Functions (string.h) continued....

Example:

```
#include<iostream>
#include<string.h>
using namespace std;
int main()
{
char first[25] = "Hello ";
char second[ ]= "World of Programming";
strncat(first, second, 6);
cout<<first<<endl;
return 0;
}
```

Output

```
Hello World
-----
Process exited after 0.03716 seconds with return value 0
Press any key to continue . . .
```

3.3 String Functions (string.h) continued....

3.3.3 strcmp()

The **strcmp()** function is used to compare two strings character by character. The comparison is **case sensitive**. It returns integer values as follows:

Returned Value	Description
Less than zero	It returns a values less than zero if the first string is less than second string.
Zero	It returns zero if the first string is equal to second string
Greater than zero	It returns a values greater than zero if the first string is greater than second string.

Syntax: **strcmp(str1, str2);**

strcmp: It is the name of the function.

str1: It indicates the first string in comparison.

str2: It indicates the second string in comparison.

3.3 String Functions (string.h) continued....

Example:

```
#include<iostream>
#include<string.h>
using namespace std;
int main()
{
char str1[ ] = "this is a test string";
char str2[ ] = "THIS IS A TEST STRING";
int result;
cout<<"Test string 1 is: "<<str1<<endl;
cout<<"Test string 2 is: "<<str2<<endl;
cout<<"Comparing string1 and string2 using strcmp ( ) ";
result = strcmp(str1, str2);
if(result==0)
    cout<<"string1 is equal to string2"<<endl;
else if (result<0)
    cout<<"string1 is less than string2"<<endl;
else
    cout<<"string1 is greater than string2"<<endl;
return 0;
}
```

Output

```
Test string 1 is: this is a test string
Test string 2 is: THIS IS A TEST STRING
Comparing string1 and string2 using strcmp()
string1 is greater than string2
```

3.3 String Functions (string.h) continued....

3.3.4 strcpy()

The word 'strcpy' stands for **string copy**. The function is used to copy one string to another including the terminating null character.

Syntax

strcpy(str1, str2);

strcpy: It is the name of the function.

str1: It indicates 1st string in which the value is to be copied.

str2: It indicates 2nd string whose value is to be copied to 1st string. It can be string variable or string constant.

3.3 String Functions (string.h) continued....

Example:

```
#include<iostream>
#include<string.h>
using namespace std;
int main()
{
char str1[50] = "Programming involves logical thinking";
char str2[50] = "Built-in functions make programming Heaven";
cout<<"str1 = "<<str1<<endl;
cout<<"str2 = "<<str2<<endl;
strcpy(str1, str2);
cout<<"After strcpy(str1, str2)"<<endl;
cout<<"str1 = "<<str1<<endl;
cout<<"str2 = "<<str2<<endl;
return 0;
}
```

Output

```
str1 = Programming involves logical thinking
str2 = Built-in functions make programming Heaven
After strcpy(str1, str2)
str1 = Built-in functions make programming Heaven
str2 = Built-in functions make programming Heaven
```

3.3 String Functions (string.h) continued....

3.3.5 strlen()

The word 'strlen' stands for **string length**. The function is used to find the length of a string. The length includes all characters including spaces in the string. It doesn't count the null character. It returns an integer value indicating the length of the string.

Syntax

strlen(str);

strlen: It is the name of the function.

str: It indicates the string whose length is to be found.

3.3 String Functions (string.h) continued....

Example:

```
#include<iostream>
#include<string.h>
using namespace std;
int main()
{
    char str[25];
    int len;
    cout<<"Enter a string: ";
    cin.getline(str,25);
    len = strlen(str);
    cout<<"The string contains "<<len<<" characters.";
    return 0;
}
```

Output

```
Enter a string: Hello World
The string contains 11 characters.
-----
```

References

1. *“C++ Programming: From Problem Analysis to Program Design”, 6/ed by D.S. Malik, (2013), CENGAGE Learning.*
2. *“Object Oriented Programming in C++,” 4/ed by Robert Lafore (2001) .*
3. *“C++ How to Program,” 5/ed by Deitel & Deitel (2005).*
4. *“Program with C++ Object Oriented Programming Aikman Series”, by C.M Aslam and T.A Qureshi.*
5. *Related documents from open source, mainly internet.*